DATA ANALYTICS FOR BUSINESS

with Lizhen Xu, Ph.D.

Introduction to R – Basics

Learning Objectives

- Assign values in R
- Perform arithmetic calculations
- Handle different types of non-numeric values
  - Characters
  - Logical values
  - Factors
- Deal with vectors in R
  - Create vectors
  - Vector indexing and subsetting
  - Vectorized operations
  - Use logical vectors for subsetting

Values, Variables, Assignment

- a <- 1.2
- b <- 3.4
- a + b
- result <- a + b

```
a <- 1.2
b <- 3.4
a + b
```

```
## [1] 4.6
```

```
result <- a + b
result
```

```
## [1] 4.6
```

Arithmetic

- 2^(2+1)-(4/3)^(1/2)
- exp(3)
- log(20.08554)
- log(x=1000, base=10)

```r
2^(2+1)-(4/3)^(1/2)
```

```
## [1] 6.845299
```

```r
exp(3)
```

```
## [1] 20.08554
```

```r
log(20.08554)
```

```
## [1] 3
```

```r
log(x=1000, base=10)
```

```
## [1] 3
```

Characters

- chr <- "Hello World!"
- nchar(chr)
- substr(chr, 1, 5)
- myfile <- "C:/test"
- setwd(myfile)

```r
chr <- "Hello World!"
nchar(chr)
```

```
## [1] 12
```

```r
substr(chr, 1, 5)
```

```
## [1] "Hello"
```

```r
myfile <- "C:/test"
setwd(myfile)
```

Logical Values

- f1 <- TRUE
- f2 <- FALSE
- f3 <- (1 > 2)
- f1 == f2
- f1 != f3
- f1 & f2
- f1 | f2
- !f3
- !f3 & (f1|f2)

```
f1 <- TRUE
f2 <- FALSE

f3 <- (1 > 2)
f3
```

```
## [1] FALSE
```

```
f1 == f2
```

```
## [1] FALSE
```

```
f1 != f3
```

```
## [1] TRUE
```

```
f1 & f2
```

```
## [1] FALSE
```

```
f1 | f2
```

```
## [1] TRUE
```

```
!f3
```

```
## [1] TRUE
```

```
!f3 & (f1|f2)
```

```
## [1] TRUE
```

Vectors (1/4)

Creating a Vector

- myvec <- 1:5
- myvec <- c(1,2,4,8,16)
- myvec2 <- seq(from=1, to=10, by=2)
- myvec3 <- rep(c(1,2,3), 5)
- length(myvec)

```
myvec <- 1:5
myvec
```

```
## [1] 1 2 3 4 5
```

```
myvec <- c(1,2,4,8,16)
myvec
```

```
## [1]  1  2  4  8 16
```

```
myvec2 <- seq(from=1, to=10, by=2)
myvec2
```

```
## [1] 1 3 5 7 9
```

```
myvec3 <- rep(c(1,2,3), 5)
myvec3
```

```
##  [1] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
```

```
length(myvec)
```

```
## [1] 5
```

Vectors (2/4)

Vector Indexing and Subsetting

- myvec[1]
- myvec[length(myvec)]
- myvec[c(1,3)]
- myvec[1:3]
- myvec[-1]
- myvec[-c(1,3)]
- index <- c(1,3,3,2,1,5,1)
- myvec[index]

- myvec[1] <- 10
- myvec[1:3] <- 10:8

```
myvec
```

```
## [1]  1  2  4  8 16
```

```
myvec[1]
```

```
## [1] 1
```

```
myvec[length(myvec)]
```

```
## [1] 16
```

```
myvec[c(1,3)]
```

```
## [1] 1 4
```

```
myvec[1:3]
```

```
## [1] 1 2 4
```

```
myvec[-1]
```

```
## [1]  2  4  8 16
```

```
myvec[-c(1,3)]
```

```
## [1]  2  8 16
```

```
index <- c(1,3,3,2,1,5,1)
myvec[index]
```

```
## [1]  1  4  4  2  1 16  1
```

```
myvec[1] <- 10
myvec
```

```
## [1] 10  2  4  8 16
```

```
myvec[1:3] <- 10:8
myvec
```

```
## [1] 10  9  8  8 16
```

Vectors (3/4)

Vectorized (Element-wise) Operations

- myvec * 2
- myvec + myvec2
- myvec <= 3
- myvec > myvec2

- sum(myvec)
- prod(myvec)

```
myvec
```

```
## [1]  1  2  4  8 16
```

```
myvec2
```

```
## [1] 1 3 5 7 9
```

```
myvec * 2
```

```
## [1]  2  4  8 16 32
```

```
myvec + myvec2
```

```
## [1]  2  5  9 15 25
```

```
myvec <= 3
```

```
## [1]  TRUE  TRUE FALSE FALSE FALSE
```

```
myvec > myvec2
```

```
## [1] FALSE FALSE FALSE  TRUE  TRUE
```

```
sum(myvec)
```

```
## [1] 31
```

```
prod(myvec)
```

```
## [1] 1024
```

Vectors (4/4)

Use Logical Vectors for Subsetting

- myvec[c(TRUE,FALSE,TRUE,FALSE,FALSE)]
- myvec[myvec > 3]

```
myvec
```

```
## [1]  1  2  4  8 16
```

```
myvec[c(TRUE,FALSE,TRUE,FALSE,FALSE)]
```

```
## [1] 1 4
```

```
myvec > 3
```

```
## [1] FALSE FALSE  TRUE  TRUE  TRUE
```

```
myvec[myvec > 3]
```

```
## [1]  4  8 16
```

Factors

- Factors are the data type used in R to represent a finite number of *distinct* categories

  - values <- c("medium", "low", "low", "high", "medium")
  - values.fac <- factor(values)
  - levels(values.fac) <- c(3,1,2)

```
values <- c("medium", "low", "low", "high", "medium")
values.fac <- factor(values)
values
```

```
## [1] "medium" "low"    "low"    "high"   "medium"
```

```
values.fac
```

```
## [1] medium low    low    high   medium
## Levels: high low medium
```

```
levels(values.fac)
```

```
## [1] "high"   "low"    "medium"
```

```
levels(values.fac) <- c(3,1,2)
levels(values.fac)
```

```
## [1] "3" "1" "2"
```

```
values.fac
```

```
## [1] 2 1 1 3 2
## Levels: 3 1 2
```

Summary

- Value assignment in R
- Arithmetic calculations
- Non-numeric values:
  - Characters
  - Logical values
  - Factors
- Handling vectors in R:
  - Create vectors
  - Vector indexing and subsetting
  - Vectorized operations
  - Logical vectors for subsetting

DATA ANALYTICS FOR BUSINESS

with Lizhen Xu, Ph.D.

Introduction to R – Data Structure

Learning Objectives

- Handle *matrices* in R
- Use *arrays* in R
- Deal with *lists* in R
- Handle *data frames* in R

Matrix (1/3)

Create a Matrix

- A <- matrix(1:12, 3, 4)

  # Note: the default is "*column-major order*"
- a1 <- 1:4
- a2 <- 5:8
- a3 <- 9:12
- M1 <- cbind(a1,a2,a3)
- M2 <- rbind(a1,a2,a3)
- dim(A)
- nrow(A)
- ncol(A)

```
A <- matrix(1:12, 3, 4)
A
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
a1 <- 1:4
a2 <- 5:8
a3 <- 9:12
M1 <- cbind(a1,a2,a3)
M1
```

```
##      a1 a2 a3
## [1,]  1  5  9
## [2,]  2  6 10
## [3,]  3  7 11
## [4,]  4  8 12
```

```
M2 <- rbind(a1,a2,a3)
M2
```

```
##    [,1] [,2] [,3] [,4]
## a1    1    2    3    4
## a2    5    6    7    8
## a3    9   10   11   12
```

```
dim(A)
```

```
## [1] 3 4
```

```
nrow(A)
```

```
## [1] 3
```

```
ncol(A)
```

```
## [1] 4
```

Matrix (2/3)

Matrix Subsetting

- A[1,2]
- A[1,]
- A[,2]
- A[,2:3]
- A[c(3,1),2:3]
- A[-1,2:3]
- A[1,] <- 11:14
- diag(A)

```
A
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
A[1,2]
```

```
## [1] 4
```

```
A[1,]
```

```
## [1]  1  4  7 10
```

```
A[,2]
```

```
## [1] 4 5 6
```

```
A[,2:3]
```

```
##      [,1] [,2]
## [1,]    4    7
## [2,]    5    8
## [3,]    6    9
```

```
A[c(3,1),2:3]
```

```
##      [,1] [,2]
## [1,]    6    9
## [2,]    4    7
```

```
A[-1,2:3]
```

```
##      [,1] [,2]
## [1,]    5    8
## [2,]    6    9
```

```
A[1,] <- 11:14
A
```

```
##      [,1] [,2] [,3] [,4]
## [1,]   11   12   13   14
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
diag(A)
```

```
## [1] 11  5  9
```

Matrix (3/3)

Matrix Operations

- A * 2
- B != 0
- A + B
- A * B # Note: * for *element-wise multiplication*
- t(B)
- A %*% t(B) # Note: %*% for *Matrix multiplication*
- c(A) # Convert a matrix to a vector

```
c(A)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12
```

```
A <- matrix(1:12, 3, 4)
A
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
B <- cbind(c(1,0,0), c(0,0,1), c(0,1,0), c(1,1,1))
B
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    0    0    1
## [2,]    0    0    1    1
## [3,]    0    1    0    1
```

```
A * 2
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    2    8   14   20
## [2,]    4   10   16   22
## [3,]    6   12   18   24
```

```
B != 0
```

```
##       [,1]  [,2]  [,3] [,4]
## [1,]  TRUE FALSE FALSE TRUE
## [2,] FALSE FALSE  TRUE TRUE
## [3,] FALSE  TRUE FALSE TRUE
```

```
A + B
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    2    4    7   11
## [2,]    2    5    9   12
## [3,]    3    7    9   13
```

```
A * B
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    0    0   10
## [2,]    0    0    8   11
## [3,]    0    6    0   12
```

```
t(B)
```

```
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    0    1
## [3,]    0    1    0
## [4,]    1    1    1
```

```
A %*% t(B)
```

```
##      [,1] [,2] [,3]
## [1,]   11   17   14
## [2,]   13   19   16
## [3,]   15   21   18
```

Arrays

- Extend matrices into multidimensions
  - An *n* x *m* x *k* (x *p*…) array has:

    n rows x *m* columns x *k* layers (x *p* blocks …)

  - AR <- array(data=1:24, dim=c(3,4,2))

    # Note: dimension: 3x4x2: 3 rows, 4 columns, 2 layers

    # Data filled into the array in *column-major order*

    # (i.e., filling the *first column* of the *first layer* first,

    # then the *second column* of the *first layer*, ...)

  - AR[,,1]
  - AR[2,,2]

```r
AR <- array(data=1:24, dim=c(3,4,2))
AR
```

```
## , , 1
##
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
##
## , , 2
##
##      [,1] [,2] [,3] [,4]
## [1,]   13   16   19   22
## [2,]   14   17   20   23
## [3,]   15   18   21   24
```

```r
AR[,,1]
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```r
AR[2,,2]
```

```
## [1] 14 17 20 23
```

Lists

- The list is a very useful data structure in R to group together *a mix of various R objects*.
- Lists are often used to store results from various R packages we will use in this course, so you will encounter lists a lot.

  - mylist <- list(matrix(1:10, 2, 5), c(T,F,T,T), "Hello World")
  - mylist[[1]]
  - mylist[c(2,3)]

```r
mylist <- list(matrix(1:10, 2, 5), c(T,F,T,T), "Hello World")
mylist
```

```
## [[1]]
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
##
## [[2]]
## [1]  TRUE FALSE  TRUE  TRUE
##
## [[3]]
## [1] "Hello World"
```

```r
mylist[[1]]
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
```

```r
mylist[c(2,3)]
```

```
## [[1]]
## [1]  TRUE FALSE  TRUE  TRUE
##
## [[2]]
## [1] "Hello World"
```

Lists (cont'd)

- names(mylist) <- c("mat", "logical", "string")
- mylist$mat
- mylist <- list(mat=matrix(1:10, 2, 5), logical=c(T,F,T,T), string="Hello World")
- mylist$logical
- mylist$vector <- 1:5

```
names(mylist) <- c("mat", "logical", "string")
mylist
```

```
## $mat
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
##
## $logical
## [1]  TRUE FALSE  TRUE  TRUE
##
## $string
## [1] "Hello World"
```

```
mylist$mat
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
```

```
mylist <- list(mat=matrix(1:10, 2, 5), logical=c(T,F,T,T),
               string="Hello World")
mylist
```

```
## $mat
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
##
## $logical
## [1]  TRUE FALSE  TRUE  TRUE
##
## $string
## [1] "Hello World"
```

```
mylist$logical
```

```
## [1]  TRUE FALSE  TRUE  TRUE
```

```
mylist$vector <- 1:5
mylist
```

```
## $mat
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
##
## $logical
## [1]  TRUE FALSE  TRUE  TRUE
##
## $string
## [1] "Hello World"
##
## $vector
## [1] 1 2 3 4 5
```

Data Frames (1/3)

- Data frames can be viewed as lists where all members must be *vectors of equal length*.
- Data frames are also like matrices, except that *columns can be in different data types*.
- Data frames are often used to *store datasets imported from external files*, so you will use data frames a lot.

Data Frames (2/3)

- mydata <- data.frame(name=c("Alice", "Bob", "Claire", "David", "Emily"), age=c(32, 40, 17, 14, 25), sex=factor(c("F","M","F","M","F")))

- mydata[2,2]
- mydata[,c(1,3)]
- mydata$name

```
mydata <- data.frame(name=c("Alice", "Bob", "Claire", "David", "Emily"),
                     age=c(32, 40, 17, 14, 25),
                     sex=factor(c("F","M","F","M","F")))
mydata
```

```
##     name age sex
## 1  Alice  32   F
## 2    Bob  40   M
## 3 Claire  17   F
## 4  David  14   M
## 5  Emily  25   F
```

```
mydata[2,2]
```

```
## [1] 40
```

```
mydata[,c(1,3)]
```

```
##     name sex
## 1  Alice   F
## 2    Bob   M
## 3 Claire   F
## 4  David   M
## 5  Emily   F
```

```
mydata$name
```

```
## [1] "Alice"  "Bob"    "Claire" "David"  "Emily"
```

Data Frames (3/3)

Use Logical Values for Record Subsetting

- mydata[mydata$sex=="M",]
- mydata[mydata$sex=="M",-3]
- mydata[mydata$sex=="M",c("name", "age")]

```
mydata
```

```
##     name age sex
## 1  Alice  32   F
## 2    Bob  40   M
## 3 Claire  17   F
## 4  David  14   M
## 5  Emily  25   F
```

```
mydata[mydata$sex=="M",]
```

```
##    name age sex
## 2   Bob  40   M
## 4 David  14   M
```

```
mydata[mydata$sex=="M",-3]
```

```
##    name age
## 2   Bob  40
## 4 David  14
```

```
mydata[mydata$sex=="M",c("name", "age")]
```

```
##    name age
## 2   Bob  40
## 4 David  14
```

Summary

- Matrices
- Arrays
- Lists
- Data Frames

DATA ANALYTICS FOR BUSINESS

with Lizhen Xu, Ph.D.

Introduction to R – Files and Controls

Learning Objectives

- Read and write *files* in R
- Install and load *packages*
- Use if-else statements for *conditions*
- Use for statements for *loops*

Reading Files

- read.table is the general command to import plain-text data files in R.
- read.csv is a simplified version of read.table to import comma-separated (csv) files.

- setwd("C:/test") # Set working directory
- mydata <- read.table("Auto.csv", header=TRUE, sep=",")
- mydata <- read.csv("Auto.csv", header=TRUE)
- fix(mydata) # To view and edit the dataset

Data Editor

File Edit Help

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin	name
1	18	8	307	130	3504	12	70	1	chevrolet chevelle malibu
2	15	8	350	165	3693	11.5	70	1	buick skylark 320
3	18	8	318	150	3436	11	70	1	plymouth satellite
4	16	8	304	150	3433	12	70	1	amc rebel sst
5	17	8	302	140	3449	10.5	70	1	ford torino
6	15	8	429	198	4341	10	70	1	ford galaxie 500
7	14	8	454	220	4354	9	70	1	chevrolet impala
8	14	8	440	215	4312	8.5	70	1	plymouth fury iii
9	14	8	455	225	4425	10	70	1	pontiac catalina
10	15	8	390	190	3850	8.5	70	1	amc ambassador dpl
11	15	8	383	170	3563	10	70	1	dodge challenger se
12	14	8	340	160	3609	8	70	1	plymouth 'cuda 340
13	15	8	400	150	3761	9.5	70	1	chevrolet monte carlo
14	14	8	455	225	3086	10	70	1	buick estate wagon (sw)
15	24	4	113	95	2372	15	70	3	toyota corona mark ii
16	22	6	198	95	2833	15.5	70	1	plymouth duster
17	18	6	199	97	2774	15.5	70	1	amc hornet
18	21	6	200	85	2587	16	70	1	ford maverick
19	27	4	97	88	2130	14.5	70	3	datsun pl510
20	26	4	97	46	1835	20.5	70	2	volkswagen 1131 deluxe sedan
21	25	4	110	87	2672	17.5	70	2	peugeot 504
22	24	4	107	90	2430	14.5	70	2	audi 100 ls
23	25	4	104	95	2375	17.5	70	2	saab 99e
24	26	4	121	113	2234	12.5	70	2	bmw 2002
25	21	6	199	90	2648	15	70	1	amc gremlin
26	10	8	360	215	4615	14	70	1	ford f250
27	10	8	307	200	4376	15	70	1	chevy c20
28	11	8	318	210	4382	13.5	70	1	dodge d200
29	9	8	304	193	4732	18.5	70	1	hi 1200d
30	27	4	97	88	2130	14.5	71	3	datsun pl510

Writing Files

- write.table is the general R command to export data into plain-text files.
- write.csv is a simplified version of write.table to export data into comma-separated (csv) files.

- write.table(mydata, file="Auto.csv", sep=",",row.names=FALSE)
- write.csv(mydata, file="Auto.csv", row.names=FALSE)

Install and Load Packages

- Specialized and useful functions and data sets are often contained in *packages* (a.k.a. *libraries*), created and shared by contributors around the world.
- To use an external package **for the first time**, one needs to first download and install it, using the command install.packages.
- To load an **installed** package, use the command library.
- To use a package, it needs to be *installed only once* on a computer, but needs to be *loaded every time* R has been restarted.

  - install.packages("rpart")
  - library(rpart)

Conditions

- We use if-else statements to run different blocks of code depending on whether a certain condition is satisfied or not.

- if (number %% 2 == 0) {
- cat("This is an even number.")
- } else {
- cat("This is an odd number.")
- }

```
number <- 3
if (number %% 2 == 0) {
  cat("This is an even number.")
} else {
  cat("This is an odd number.")
}
```

```
## This is an odd number.
```

Loops

- We use for statements to repeatedly run a chunk of code for a given number of times as controlled by the index (a.k.a. counter).

- myvec <- 1:100
- sum <- 0
- for (i in 1:length(myvec))
- {
- sum <- sum + myvec[i]
- }

```
myvec <- 1:100
sum <- 0
for (i in 1:length(myvec))
{
  sum <- sum + myvec[i]
}
sum
```

```
## [1] 5050
```

Summary

- Read and write *files* in R
  - read.table, read.csv
  - write.table, write.csv
- Install and load *packages*
  - install.packages
  - library
- Use if-else statements for *conditions*
- Use for statements for *loops*

DATA ANALYTICS FOR BUSINESS

with Lizhen Xu, Ph.D.

Introduction to R – Functions and Visualization

Learning Objectives

- Master different ways of calling a function
  - Exact matching
  - Partial matching
  - Positional matching
  - Mixed matching
- Perform basic data visualization
  - Histogram
  - Bar Charts
  - Box Plots
  - Scatterplots

Calling Functions

- We need to pass in values to *arguments* when calling a function.
- To see the argument list of a function, use ? or args
  - ?matrix
  - args(matrix)

```
args(matrix)
```

```
## function (data = NA, nrow = 1, ncol = 1, byrow = FALSE, dimnames = NULL)
## NULL
```

Calling Functions: Exact Matching

- Different ways of ***argument matching*** in R
- **Exact matching:** Spell out each argument tag in full.

  Note: *The order of the argument values doesn't matter in this case.*

  - A <- matrix(data=1:12, nrow=3, ncol=4)
  - A <- maxtrix(nrow=3, ncol=4, data=1:12)

```
matrix(data=1:12, nrow=3, ncol=4)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
matrix(nrow=3, ncol=4, data=1:12)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

Calling Functions: Partial Matching

- **Partial matching:** Only spell out the first several letters of each argument tag such that each abbreviated tag is unique.

  Note: Order still doesn't matter.

  - A <- matrix(dat=1:12, nr=3, nc=4)
  - A <- matrix(nr=3, nc=4, dat=1:12)

```
matrix(dat=1:12, nr=3, nc=4)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
matrix(nr=3, nc=4, dat=1:12)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

Calling Functions: Positional Matching

- **Positional matching:** Supply argument values without tags, identified solely based on the order.

  Note: Order matters! Must follow the same order of arguments as in function definition.

  - A <- matrix(1:12, 3, 4)
  - A <- matrix(1:12, 3, 4, TRUE)

```
matrix(1:12, 3, 4)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
matrix(1:12, 3, 4, TRUE)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8
## [3,]    9   10   11   12
```

Calling Functions: Mixed Matching

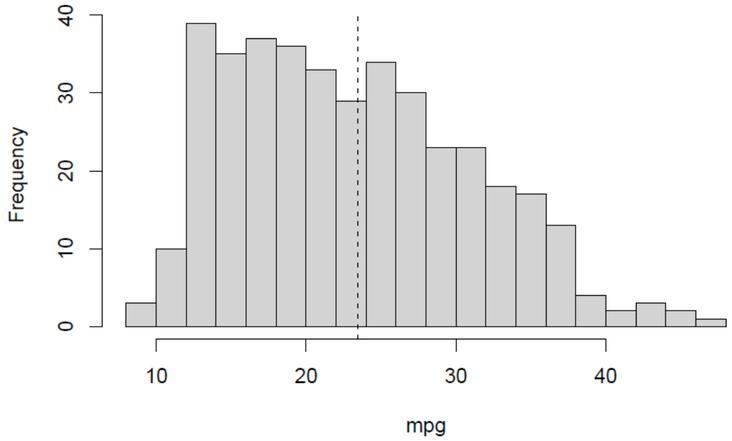
- **Mixed matching:** Mix positional and exact/partial matching
  - A <- matrix(1:12, 3, 4, dimnames=list(c("A","B","C")))

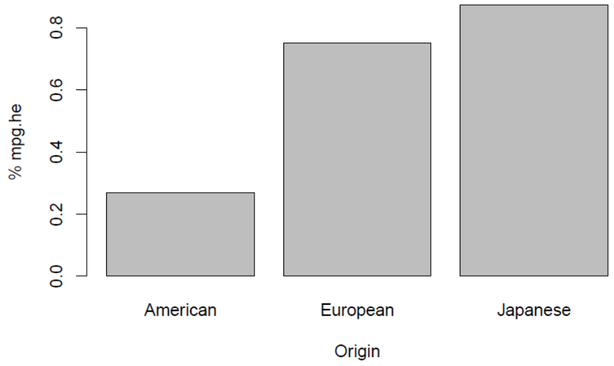
```
matrix(1:12, 3, 4, dimnames=list(c("A","B","C")))
```

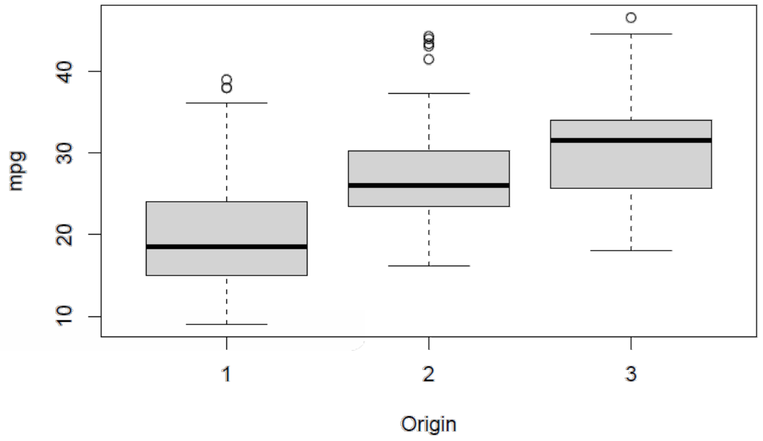
```
##   [,1] [,2] [,3] [,4]
## A    1    4    7   10
## B    2    5    8   11
## C    3    6    9   12
```

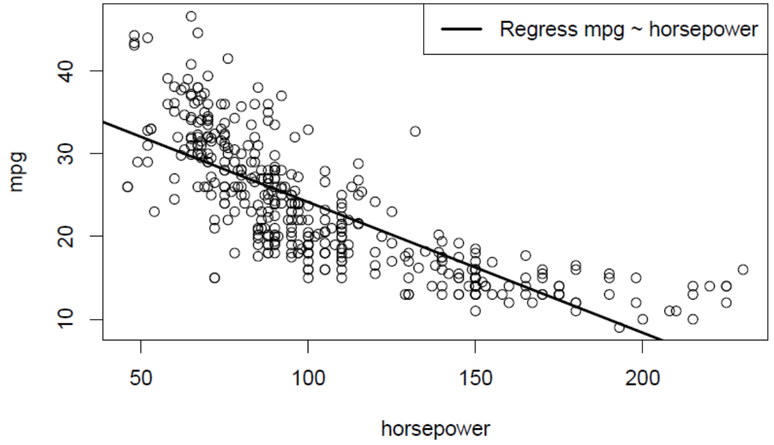
Basic Data Visualization

- Histogram
- Bar Charts
- Box Plots
- Scatterplots









Summary

- Different ways of argument matching when calling a function:
  - Exact matching
  - Partial matching
  - Positional matching
  - Mixed matching

- Basic data visualization:
  - Histogram
  - Bar Charts
  - Box Plots
  - Scatterplots